

Extended Abstract

Motivation Lower-limb exoskeletons are increasingly being used to enhance human mobility by applying assistive torques at the joints. While effective in steady-state conditions, current low-level control strategies, such as proportional-derivative (PD) controllers with or without iterative learning (PD+LRN), often fail to adapt to walking bouts that feature rapid changes in gait speed or non-periodic motion. These limitations stem from their reliance on static gains or stride-to-stride similarity, which do not generalize well to real-world gait variability. This project investigates whether reinforcement learning (RL) can enhance torque-tracking robustness by learning a velocity compensation policy that generalizes across different walking speeds and conditions.

Method We developed a residual RL controller to augment a baseline PD policy in tracking desired ankle torques during human-like walking. Desired torque profiles were parameterized by four key timing and magnitude parameters and synthesized across a range of walking speeds using empirical relationships with added variability. Each walking bout consisted of 40 gait cycles, with joint kinematics and desired torque trajectories sampled at 30 time steps per cycle. The control policy was trained using Proximal Policy Optimization (PPO), where the state included joint positions, velocities, speed, and desired torque. The scalar action represented a residual motor velocity command. Two reward functions were tested: one based solely on torque error, and another that also penalized residual effort.

Implementation The control environment was implemented in MuJoCo, modeling a cable-driven ankle exoskeleton inspired by Zhang et al. (2017, 2021). A virtual spring linked the pulley to the ankle joint, simulating elastic transmission. A 20 ms delay was introduced to represent actuation lag. PPO was configured with a 2-layer MLP (128 units per layer), a learning rate of 3×10^{-5} , 2048 steps per update, and 1 million training time steps. Training was conducted locally using pre-generated gait data.

Results The PD+RL controller trained with the combined torque error and residual penalty reward outperformed both PD and PD+LRN in terms of average RMS torque error across test strides (0.281 vs. 0.308 and 0.382, respectively). It also achieved the lowest average stride RMSE. However, the version trained without the residual penalty exhibited higher tracking variance and occasional overshooting, highlighting the importance of reward design. Despite these improvements, the PD baseline remained highly competitive and more consistent across conditions. The PD+LRN controller performed worst, particularly under irregular gait, confirming its dependence on stride periodicity.

Discussion While PD+LRN is effective under cyclic conditions, its performance degrades when stride consistency is lost. RL-based controllers showed potential to generalize better to non-cyclic walking, particularly when residual effort was regularized. However, the current design lacked access to temporal context or gait phase information, limiting its ability to time corrections effectively. Furthermore, the reward function—focused on minimizing pointwise torque error—did not encourage precise phase alignment or stability during transitions.

Conclusion Residual RL is a promising strategy for augmenting low-level torque controllers in variable-speed walking. Our results show that with an appropriately shaped reward, RL can outperform traditional methods in average tracking performance. Nonetheless, the controller’s variance and reliance on stateless observations limit its robustness. Future work should explore phase-aware state representations, more expressive policy architectures, and reward functions that explicitly account for stability and coordination.

Learning Torque-Tracking Control for Speed-Adaptive Exoskeletons

Maisha Khanum

Department of Mechanical Engineering
Stanford University
mkhanum@stanford.edu

Abstract

This work investigates the use of reinforcement learning (RL) to improve low-level torque tracking in ankle exoskeletons during variable-speed, non-periodic walking. We simulate an exoskeleton with a spring-motor system and train a residual RL policy to augment a baseline PD controller. Our results show that a reward function combining torque tracking accuracy and residual effort penalty enables the RL policy to outperform PD and PD+LRN baselines in average tracking error. However, the policy remains sensitive to gait timing and exhibits higher variance without phase or memory information. These findings underscore the potential of residual RL for adaptive exoskeleton control and the importance of reward design and temporal context in unstable gait scenarios.

1 Introduction

Ankle exoskeletons are a promising tool for augmenting human walking, particularly in elderly or gait-impaired individuals. Torque assistance is most effective when personalized to the user and adaptive to changing conditions. Existing controllers are tuned for steady-state walking and often break down when walking speed changes dynamically. This project explores whether reinforcement learning can provide real-time compensation for such speed-related disturbances.

2 Related Work

PD-based low-level controllers have historically been effective for steady-state walking tasks due to their simplicity, robustness, and low computational cost. They are particularly well-suited for cyclic, predictable motion Zhang et al. (2017). However, in the context of untethered ankle exoskeletons, this simplicity becomes a limitation. These systems typically use Bowden cable transmissions and spring-loaded mechanisms, which behave like series elastic actuators. The passive compliance introduced by this transmission makes accurate torque tracking difficult under dynamic gait conditions.

This challenge is highlighted by Zhang and Collins (2021), which shows that the mechanical stiffness between the motor and the joint dramatically influences tracking performance. While lower stiffness allows for more shock absorption and compliance, it also increases sensitivity to time delays and motor velocity error, making conventional PD controllers insufficient under variable conditions.

To address this, Zhang et al. (2017) proposed combining PD control with iterative learning (PD+LRN), enabling the controller to adapt a learned feedforward signal over successive strides. This method exploits the periodicity of walking to improve torque tracking over time. However, it assumes cyclicity, and its effectiveness degrades with irregular or transitioning gait.

At a higher level, human-in-the-loop optimization (HILO) has become a popular approach for tuning assistance parameters to minimize metabolic cost during walking. Slade et al. (2024) demonstrated

how online optimization can personalize exoskeleton torque profiles in real-world settings. However, these methods operate on slow time scales and are too reactive to compensate for stride-to-stride disturbances. As real-world walking is highly non-periodic, due to changes in speed, terrain, and user intent, there is growing interest in controllers that can adapt rapidly and autonomously. Improving low-level torque controllers is therefore critical to enabling robust performance across walking conditions.

Reinforcement learning (RL) offers a promising alternative due to its ability to learn from interaction and adapt to new situations. RL has been applied extensively to whole-body locomotion in bipedal robots, particularly for trajectory generation and stabilization Schulman et al. (2017). However, its use in augmenting low-level torque control for wearable devices remains underexplored.

One promising framework is residual reinforcement learning, where RL is used to learn a corrective term on top of a stable base controller. This technique was introduced in Johannink et al. (2018), where the learned policy augments a PID or PD controller by issuing small adjustments to improve performance. Deisenroth and Rasmussen (2017) applied this concept to velocity compensation in a vehicle suspension system, where a residual policy learned to improve upon a PID baseline. These studies show that residual learning can be particularly effective in scenarios with structured prior knowledge, as it preserves stability while enhancing adaptivity.

In this work, we build on the residual RL paradigm by learning a scalar motor velocity compensation term to augment a PD torque controller in a cable-driven ankle exoskeleton. To our knowledge, this is one of the first applications of residual RL in the context of low-level torque control for lower-limb assistive devices.

Our dataset was grounded in biomechanics literature. Burssens et al. (2024) documented strong correlations between walking speed and ankle joint kinematics, showing that peak dorsiflexion and angular velocity vary predictably with speed. We interpolated these relationships and used them to generate speed-dependent ankle trajectories. To simulate realistic gait variation, we introduced randomized perturbations in stride timing and joint profiles.

Returning to human-in-the-loop optimization, Slade et al. (2022) demonstrated a clear relationship between walking speed and the optimal torque profile parameters (e.g., peak time, magnitude, rise/fall time). This finding emphasizes that even in high-level personalization schemes, low-level torque controllers must be capable of adapting to speed changes to apply optimal assistance at the correct time.

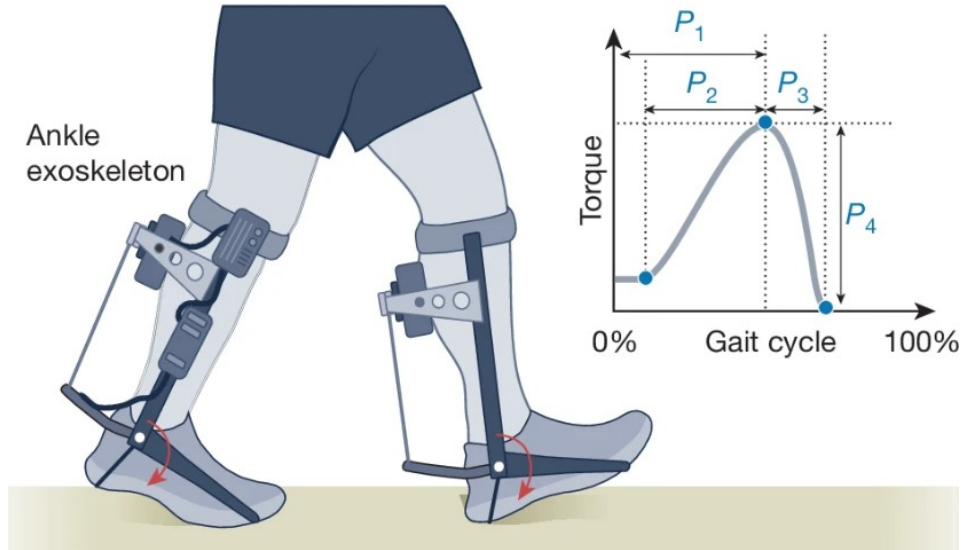


Figure 1: Speed-dependent optimal torque parameters from Slade et al. (2022).

While prior work has established effective methods for high-level adaptation and low-level control in steady gait, few have addressed the need for robust, adaptive low-level controllers for exoskeletons in

non-periodic, variable-speed walking. This motivates our exploration of residual RL to augment a classical torque tracking controller.

3 Method

We modeled the ankle exoskeleton as a spring-motor transmission system, drawing inspiration from previous work by the model from Zhang et al. (2017) seen in Figure 2. The system consists of a brushless DC motor driving a drum connected to the ankle via a Bowden cable. The compliance introduced by the cable results in behavior similar to a series elastic actuator, which complicates torque control due to the indirect relationship between motor input and joint output. We opted not to implement any high-level controller logic to focus on the impact of the low-level controller.

3.1 Mechanical Model

The ankle torque is generated by deflection across the cable transmission. Assuming negligible friction and no cable slack, the applied ankle torque τ_a is:

$$\tau_a = K_t(R\theta_p - \theta_e) \quad (1)$$

where θ_p is the motor pulley angle, θ_e is the ankle joint angle, $R = r_p/r_a$ is the ratio of pulley radius to ankle moment arm, and $K_t = r_a^2 K_c$ is the effective stiffness derived from cable stiffness K_c . Following Slade et al. (2022), we set $r_a = 0.115$ m, $r_p = 0.020$ m, and $K_c = 195$ N/m, giving:

$$R = 0.174, \quad K_t = (0.115)^2 \cdot 195 = 2.577 \text{ Nm/rad}$$

Thus, the ankle torque becomes:

$$\tau_a = 2.577 \cdot (0.174 \cdot \theta_p - \theta_e) \quad (2)$$

This nonlinear stiffness mapping motivates the need for accurate control of motor displacement to track desired torque profiles. A major assumption in this system is the linearity of the cable transmission, which, in the real system, is subject to nonlinearity due to stiffening from varying loads and temperatures. Beyond this simulation experiment, this methodology should be tested on the real system to get fully accurate conclusions.

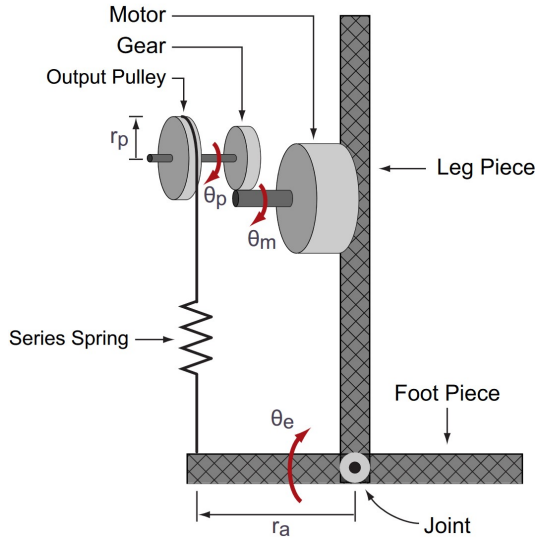


Figure 2: Schematic of one degree-of-freedom ankle exoskeleton with a series elastic actuator. From Zhang, 2017.

3.2 Baseline Controllers

Proportional-Derivative (PD) Controller. We selected a PD controller as the conventional baseline due to its widespread use in embedded exoskeleton systems. Its structure is simple and computationally efficient:

$$\Delta\theta_{p,\text{des}} = -K_p e_\tau - K_d \dot{\theta}_p \quad (3)$$

where $e_\tau = \tau_a - \tau_{\text{des}}$ is the torque error and $\dot{\theta}_p$ is the motor angular velocity. This command is integrated and applied as a velocity command at 20 Hz. A control delay of 0.02 s is imposed to emulate real actuator behavior.

PD+LRN (Iterative Learning). Iterative learning was proposed by Zhang et al. (2017) to improve tracking during cyclic tasks like walking. By maintaining a stride-indexed buffer of prior errors, the controller refines feedforward terms across cycles:

$$\Delta\theta_{p,\text{des}}^{LRN}(i, n+1) = \beta \cdot \Delta\theta_{p,\text{des}}^{LRN}(i, n) - K_l \cdot e_{fit}(i, n) \quad (4)$$

$$\Delta\theta_{p,\text{des}}(i, n) = -K_p e_\tau(i, n) - K_d \dot{\theta}_p(i, n) + \Delta\theta_{p,\text{des}}^{LRN}(i + D, n) \quad (5)$$

This approach is highly effective for steady-state walking but breaks down when stride-to-stride variability increases.

3.3 Residual RL Controller

We augment the PD controller with a learned velocity residual u_{RL} using Proximal Policy Optimization (PPO). The final control command is:

$$u = u_{PD} + u_{RL} \quad (6)$$

The policy receives a 7D observation vector:

$$s_t = [\theta_p, \dot{\theta}_p, \theta_e, \dot{\theta}_e, \tau_{\text{des}}, \tau_{\text{meas}}, v_{\text{gait}}]$$

The observation vector $s_t \in \mathbb{R}^7$ contains:

- θ_p : pulley joint angle
- $\dot{\theta}_p$: pulley joint velocity
- θ_e : ankle joint angle
- $\dot{\theta}_e$: ankle joint velocity
- τ_{des} : desired ankle torque at current timestep
- τ_{meas} : measured ankle torque at current timestep
- v_{gait} : walking speed for the current stride

This state representation was chosen to capture a diverse set of signals while keeping a minimal number of components to speed up training. User speed is the only non-determinate component since it must be inferred externally, possibly with an IMU-based sensor. However, from our biomechanics review, walking speed is a major factor in gait change, thus there is a benefit of including it in the training and deployment.

3.4 Gain Parameter Tuning

PD gains were tuned before running the experiments. The values were kept consistent across controllers for consistency.

K_p	K_d	K_t	$\mathbf{D}$
0.81	0.70	1.95	0.02s

3.5 Choice of RL Algorithm

We selected Proximal Policy Optimization (PPO) due to its strong empirical stability and sample efficiency in continuous control domains Schulman et al. (2017). Unlike off-policy methods such as DDPG or SAC, PPO works well with on-policy rollouts, which fits our episodic learning setup where each episode is a fixed stride from a synthetic dataset. Its use of clipped surrogate objectives helps prevent destructive policy updates, which is important given the tight coupling between control action and joint torque via spring dynamics.

The policy network is a multilayer perceptron (MLP) with two hidden layers of 128 units each, using ReLU activations. This architecture is common for low-dimensional continuous control and provides sufficient representational capacity to approximate the nonlinear residual needed to complement the PD controller. We opted against a convolution neural network framework to avoid overfitting, given the relatively small state and action spaces.

3.6 Reward Design

The reward function is composed of one or two terms:

$$r_1 = -\lambda_1(\tau_a - \tau_{\text{des}})^2 \quad (7)$$

$$r_2 = -\lambda_1(\tau_a - \tau_{\text{des}})^2 - \lambda_2 u_{RL}^2 \quad (8)$$

- The primary term penalizes squared torque error, encouraging the policy to track the reference profile. The initial experiments had this term as the sole contributor to the reward function. We chose $\lambda_1 = 0.1$ to balance trying out different compensation strategies with minimizing tracking error.
- The residual effort penalty ($\lambda_2 u_{RL}^2$) discourages excessive actuation, especially since learned corrections are intended to be small adjustments rather than full replacements for PD behavior. We chose $\lambda_2 = 0.1$ to balance this penalty against tracking accuracy.

Together, these modeling decisions ensure that the RL agent learns to refine, rather than override or overshoot, the conventional control from the PD controller. The separation of base control and learned residual preserves physical stability while enabling the system to adapt to future additional nonlinearities, timing offsets, or unmodeled dynamics in human-exoskeleton interaction.

4 Experimental Setup

4.1 Gait Dataset Generation

A synthetic dataset of 1500 walking bouts was created in MATLAB. Each bout consists of 40 strides with 30 time steps per stride. Gait speeds were sampled from the range [0.8, 1.6] m/s.

- Ankle kinematic trajectories were scaled sinusoidal functions based on digitized data from Burssens et al. (2024).
- Torque profiles were parameterized by peak time, rise time, fall time, and magnitude, all made speed-dependent based on Slade et al. (2022).
- Controlled random noise was added to timings and magnitudes to ensure stride-to-stride variation.

The training dataset contained the first 1000 of these samples and was used to train the RL-based controller. The validation dataset contained the remaining 500 samples and was used to compare the policy with the baseline controllers.

4.2 MuJoCo Environment and Control Loop

We implemented a custom Gym environment wrapping MuJoCo, simulating the exoskeleton actuator and tracking torque from synthetic gait data. For the XML model, we had a general actuator serve as the rotational spring between pulley and ankle to model the series elastic actuator. Gravity was disabled and no contact forces were included to isolate torque tracking. Motor limits were enforced on velocity and torque. Each episode simulates a full stride from the dataset. A simplified spring model connects the pulley and ankle joints. The environment uses:

- 1000 Hz simulation frequency (MuJoCo internal)
- 20 Hz control updates (external loop)
- 0.02s actuator delay emulated via buffer

Consistent with the mechanical model, the torque tracking error is computed as:

$$\tau_{\text{actual}} = K_t(\theta_p - \theta_e), \quad e_\tau = \tau_{\text{actual}} - \tau_{\text{des}}$$

Episode termination occurs after 30 steps (one stride), consistent with the structure of the gait dataset.

4.3 Controller Integration

The PDResidualController integrates the classical and learned control terms. It also manages the residual buffer, computes joint torques, and outputs motor commands. By abstracting the control logic, this class enables modular training and testing of RL-enhanced controllers under realistic actuator constraints.

Together, this method evaluates whether residual RL can learn meaningful corrections on top of a conventional PD policy in a realistic, biomechanically inspired torque control task.

4.4 RL Training Configuration

We used Stable-Baselines3’s PPO with:

- **Network:** MLP (2 layers, 128 units each)
- **Learning Rate:** 3×10^{-5}
- **Batch Size:** 64
- **Steps per Update:** 2048
- **Total Timesteps:** 1 million
- **Clip Range:** 0.1, **Entropy Coefficient:** 0.01

Training was performed entirely in simulation. Each rollout simulated one full walking bout. Actions were clipped, and delay of 0.02s was implemented in control loops.

5 Results

This section evaluates the performance of PD, PD+LRN, and PD+RL controllers in terms of torque tracking accuracy, stride-to-stride consistency, qualitative behavior, and why the RL-based controller failed to generalize. In general, we see that the performance of the PD+LRN controller is very dependent on the reward terms. However, there is additional nuance when viewing the torque error profile over the gait cycle.

5.1 Quantitative Evaluation

Table 1 summarizes the root-mean-squared (RMS) torque error across all evaluation bouts. The *mean RMS error* metric reflects the average of individual stride-wise RMS errors and thus captures both accuracy and consistency across strides. The baseline PD controller achieved strong performance

with a mean RMS error of 0.308 and the lowest standard deviation (0.026), indicating consistent behavior across varying gait speeds and stride conditions. The PD+LRN controller produced the highest error (0.382) but similarly low variance, suggesting that its learned feedforward terms were stable yet poorly generalized to unseen gait profiles.

The PD+RL controller trained with the simple torque-tracking reward (r_1) slightly improved over PD+LRN but exhibited greater variability (0.026), likely due to unstable or overly aggressive corrections. In contrast, the PD+RL controller trained with the combined torque and residual penalty reward (r_2) achieved the **lowest mean RMS error (0.281)** and comparable variance, demonstrating that regularizing residual effort leads to more efficient and generalizable corrections.

To further assess per-stride behavior, we also compute the *average stride RMSE*, shown in Figure 3b. This metric represents the RMSE of the mean measured torque trace against the mean desired torque trace across all strides. It captures systematic bias or lag in the controller’s overall behavior, independent of stride-level variance. Again, PD+RL (r_2) achieves the best performance (0.271), while PD+LRN shows the worst (0.375). The consistency between both metrics confirms that the improvements in PD+RL (r_2) are not due to chance on a subset of strides, but reflect more general improvements in tracking precision and timing.

Table 1: Mean and standard deviation of RMS torque error across test bouts

Controller	Mean RMS Error (Nm)	Std Dev	Notes
PD	0.308	0.026	Most consistent across conditions
PD + LRN	0.382	0.023	Stable but poorly generalized
PD + RL (r_1)	0.364	0.026	High variance, sometimes overshoots
PD + RL (r_2)	0.281	0.024	Best performance with smooth corrections

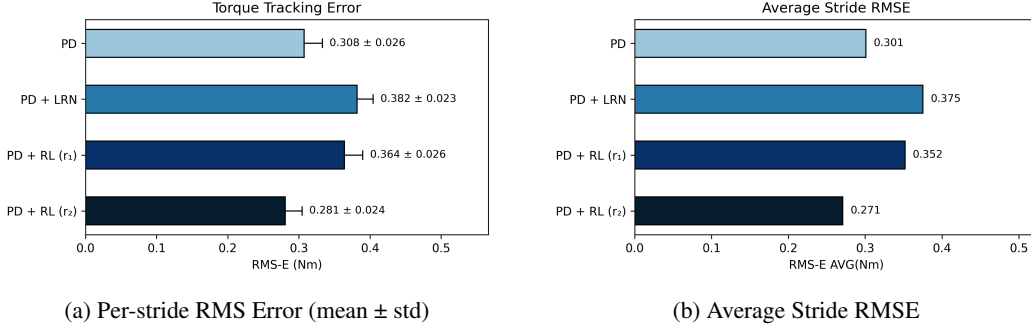


Figure 3: Torque tracking performance across controllers. Left: mean RMS error computed per stride; Right: RMSE of averaged stride.

Reward Function Analysis. Figure 4 compares the training reward trajectories for the two RL reward formulations. The torque-only reward (r_1) leads to faster convergence, reaching steady reward within 200k steps, but plateaus at a suboptimal policy. In contrast, the combined reward (r_2) converges more slowly but ultimately produces better tracking behavior and a smoother policy. This suggests that while r_1 is easier to optimize, r_2 leads to more generalizable and stable control.

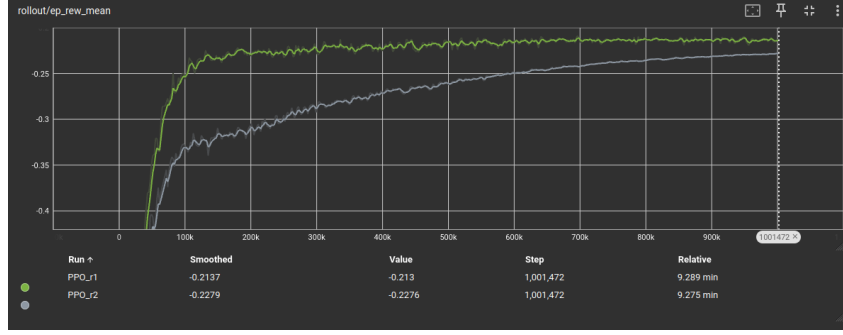


Figure 4: Training reward curves for PD + RL policies. r_1 uses only torque error; r_2 adds a residual effort penalty.

5.2 Qualitative Analysis

Figure 5 shows desired and measured torque profiles for one representative stride across all controllers. The PD controller tracks the trajectory well but lags behind during the torque rise phase due to system delay. PD + LRN exhibits smoother but consistently underpowered responses, likely due to stale feedforward terms. PD + RL (r_1) produces stronger corrections, occasionally overshooting the desired peak, while PD + RL (r_2) shows more moderated torque behavior, staying closer to the desired trajectory across all timesteps.

These observations align with the reward formulations: r_1 encourages minimizing tracking error at all costs, often producing aggressive residuals, while r_2 balances tracking with actuation effort, leading to smoother policies. Notably, the residual penalty in r_2 results in torque profiles that better preserve the physical behavior of the base PD controller.

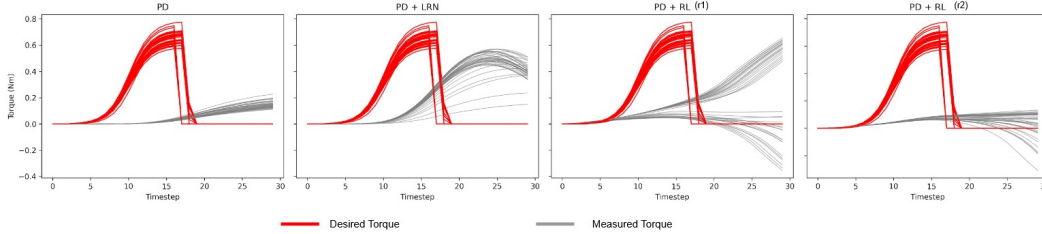


Figure 5: Desired (red) and measured (gray) torque over 30 timesteps for a representative stride under each controller.

5.3 Analysis of RL Policy Failures

The PPO-trained residual policy outputs a single scalar action per timestep, without access to past state history or an explicit phase variable. As a result, it lacks the temporal context needed to consistently apply corrections at the appropriate points in the gait cycle. This limitation is especially problematic given the episodic training setup, where each stride is treated independently. Such structure prevents the policy from learning long-term dependencies or stride-to-stride adaptation.

While the reward function penalized torque tracking error, the inclusion of a residual effort term in the r_2 formulation further constrained the policy’s corrective behavior. This penalty encouraged the agent to minimize residual amplitude even when larger interventions might have improved tracking, particularly during the peak torque region of the stride. Consequently, the policy often undercorrected in high-demand phases, leading to lower tracking error variance but also a reduced responsiveness in dynamic gait segments.

The 1D action space, while simplifying the control problem, also limited the expressiveness of the residual policy. Without access to joint-phase or a more structured policy representation (e.g., a phase-conditioned network or recurrent memory), the agent could not produce nuanced, phase-specific corrections.

These constraints explain the mixed performance observed in the results. While the PD + RL (r_2) controller ultimately achieved the best average tracking error, its corrections were modest and conservative. In contrast, the PD + RL (r_1) controller demonstrated more aggressive behaviors, but with higher variability and frequent overshoots. The timing mismatch and limited corrective authority, especially in non-cyclic or irregular gait segments, remain key contributors to the residual policy’s inconsistency and highlight the limitations of stateless, single-output architectures in time-sensitive control tasks.

6 Discussion

6.1 Dataset and Task Limitations

The synthetic gait dataset contained significant stride-to-stride variation in torque profile timing and magnitude. While realistic, this variability made learning generalizable residual corrections challenging without additional contextual input. A more structured dataset or inclusion of gait phase markers could improve learning.

Moreover, each episode simulates only a single stride, limiting the temporal context available to the policy. Thus, the impact of the simulated velocity profile across the walking bout is not explored in this project. This design choice simplifies rollout management but hinders the agent’s ability to exploit stride-level correlations.

6.2 Broader Impact and Future Work

This study underscores that while RL has the potential to augment classical control, naive implementations often underperform in high-variance conditions. Future improvements should focus on phase-aware observation design, expanded action expressiveness, and structured training curricula to guide learning. Imitation learning may also help to initialize policies from strong torque tracking examples.

6.3 Practical Challenges

Stabilizing PPO under variable stride conditions required careful tuning. Generating a diverse yet realistic gait dataset in MATLAB demanded precise parameter scaling and noise control. Ensuring MuJoCo simulation stability at high control frequencies and implementing actuation delay also introduced development complexity.

Key insight: Residual RL requires temporal awareness, non-aggressive reward structure, and biomechanical priors to succeed in variable-speed torque tracking for exoskeletons.

7 Conclusion

Learning-based controllers offer a promising alternative to hand-tuned feedback laws for exoskeleton control under variable locomotion conditions. However, our **results show that reinforcement learning-based residual control does not consistently outperform the baseline PD controller in terms of RMS torque tracking error**. While the PD+RL controller trained with a residual effort penalty (r_2) achieved the lowest average RMS error across all strides, the baseline PD controller remained competitive, particularly in terms of consistency and generalization. The PD+LRN controller, which relies on iterative error correction, performed worst overall. This is likely due to the highly non-cyclic and irregular gait patterns in our dataset, which did not allow the iterative learning term to converge.

These findings suggest that while residual RL can improve performance in some conditions, its effectiveness is highly sensitive to reward design and input structure. Our current reward functions, which focused on minimizing instantaneous torque error and residual magnitude, do not account for phase alignment, timing delay, or long-term correction trends. Without access to memory or phase information, the policy struggles to apply well-timed corrections, limiting its generalization to dynamically changing gaits. Future work should explore phase-aware inputs and multi-objective reward structures that jointly optimize for tracking accuracy, timing, and energy efficiency.

8 Team Contributions

- **Maisha Khanum:** Project conception, dataset generation, simulation model implementation, PPO training, result analysis, and full report and poster preparation.

Changes from Proposal I changed my project idea significantly from my proposal due to discovering issues with my previous problem formulation and the timeline needed to get my data collection working. I still wanted to focus on speed-adaptive exoskeleton and pivoted from context-aware stair approach problem to a low-level controller problem.

References

- A. Burssens, B. Vanwanseele, I. Jonkers, and F. De Groote. 2024. Quantifying Walking Speeds in Relation to Ankle Biomechanics on a Real-Time Interactive Gait Platform: A Musculoskeletal Modeling Approach in Healthy Adults. *Journal of Biomechanics* (2024). In press.
- M. P. Deisenroth and C. E. Rasmussen. 2017. Optimising PID Control with Residual Policy Reinforcement Learning. In *Proceedings of the 2017 Conference on Robot Learning (CoRL)*. <https://proceedings.mlr.press/v78/deisenroth17a.html>
- J. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, E. Lee, S. Levine, C. Finn, J. A. Hwangbo, S. Schaal, K. Kumar, B. Thananjeyan, C. G. Atkeson, and S. Levine. 2018. Residual Reinforcement Learning for Robot Control. <http://arxiv.org/abs/1812.06298>. arXiv preprint arXiv:1812.06298.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347* (2017). <https://doi.org/10.48550/arXiv.1707.06347> Submitted 20 Jul 2017; revised 28 Aug 2017.
- P. Slade, C. G. Atkeson, J. M. Donelan, H. Houdijk, K. A. Ingraham, M. Kim, K. Kong, K. L. Poggensee, R. Riener, M. Steinert, J. Zhang, and S. H. Collins. 2024. On Human-in-the-Loop Optimization of Human–Robot Interaction. *Nature* 633 (2024), 779–788. <https://doi.org/10.1038/s41586-024-07697-2>
- P. Slade, M. Kim, K. A. Ingraham, H. Houdijk, J. M. Donelan, and S. H. Collins. 2022. Personalizing Exoskeleton Assistance While Walking in the Real World. *Nature* 610, 7931 (2022), 277–282. <https://doi.org/10.1038/s41586-022-05191-1>
- J. Zhang, C. C. Cheah, and S. H. Collins. 2017. Torque Control in Legged Locomotion. In *Bio-Inspired Legged Locomotion: Models, Concepts, Control and Applications*, M. Sharbafi and A. Seyfarth (Eds.). Butterworth-Heinemann, Chapter 5, 347–395.
- J. Zhang and S. H. Collins. 2021. The Iterative Learning Gain That Optimizes Real-Time Torque Tracking for Ankle Exoskeletons in Human Walking Under Gait Variations. *Frontiers in Neuro-robotics* 15 (2021), 65.

A Additional Experiments

We attempted additional reward terms penalizing velocity change rate. These modifications did not significantly improve learning.

B Implementation Details

MuJoCo model and training code are implemented in Python. Gait dataset was generated in MATLAB and stored as a .MAT file. Code available in the linked Github repo: <https://github.com/maisha-khanum/ankle-torque-tracking>.